



Database

data persistence

files

What is Database?

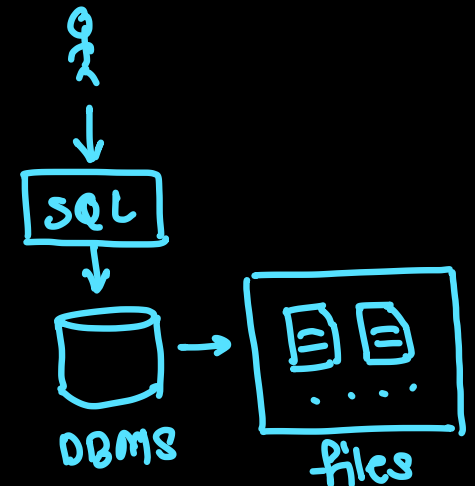
structured RDBMS → table
NoSQL → document / column / graph

- A database is an organized collection of data that can be easily accessed, managed, and updated
- Databases are used to store structured information in electronic format so that a computer program can perform various operations on the data, such as create, read, update, and delete
- The primary purpose of a database is to store large amounts of data in an organized manner and make it available to users

C → Create
U → Update
R → Read
D → Delete

Requirements

- structured information
- performance



History



- **1960:** Charles Bachman introduced Integrated Data Store (IDS), the first commercial database management system.
- **1964:** Edgar F. Codd published "A Relational Model of Data for Large Shared Data Banks," laying the theoretical foundation for relational databases.
- **1970:** IBM's System R was developed, introducing the concept of SQL (Structured Query Language).
- **1980s:** Following Codd's work, the standards for relational databases began to take shape. SQL became the de facto language for managing relational data.
- **1986:** The ANSI/ISO SQL standard was published, providing a common framework for database systems.
- **Late 1980s:** As businesses grew, the need for managing and analyzing large volumes of data became evident. Data warehousing emerged as a means to consolidate and analyze historical data from various sources. → huge amount data
- **1990s:** Companies like Teradata, Informatica, and Microsoft developed specialized database systems for data warehousing
- **2009:** "NoSQL" was coined to describe a new class of databases that moved away from the traditional relational model, focusing on scalability, availability, and performance.
- **2010s:** New technologies like MongoDB (document store), Cassandra (wide-column store), Redis (key-value store), and Couchbase (document-oriented) gained popularity.

Popular Databases



- 1970: IBM DB2 was developed by Charles Bachman
- 1977: Oracle Database was developed by Larry Ellison, Bob Miner, and Andy Bechtolsheim
- 1980: Informix was developed by Eric Epstein, Daniel M. Ross
- 1985: Sybase was developed by Roger Lemer, Bruce Lindsay
- 1986: Microsoft SQL Server was developed by Paul
- 1986: PostgreSQL was developed by Jürgen Schmitz, Mike Stonebraker
- 1995: MySQL was developed by David Axmark, Allan Larsson, and Michael Widenius
- 2009: MariaDB was developed by Michael Widenius (forked from MySQL)
- 2009: Amazon RDS was developed by Amazon Web Services

Key Components

■ Data

- The raw facts and figures stored

■ Schema

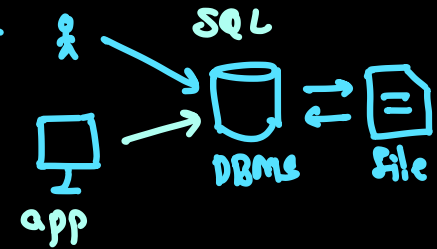
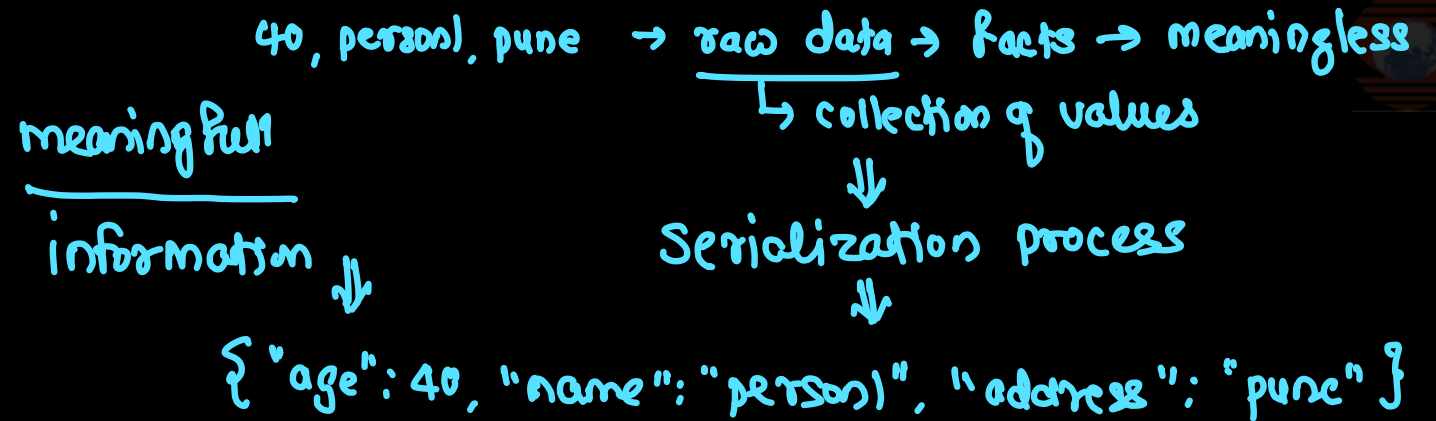
- The structure of the database – defines data types, relationships, and constraints. It's like a blueprint for your data → tables, functions, triggers, database

■ DBMS (Database Management System)

- Software that allows users to define, create, query, update, and administer databases
- Examples: MySQL, PostgreSQL, Oracle, Microsoft SQL Server, MongoDB (NoSQL)

■ Query Language → SQL

- A language used to communicate with the DBMS and retrieve/manipulate data
- The most common is SQL (Structured Query Language)



Types of Databases (DBMS)

■ Relational Databases (RDBMS)

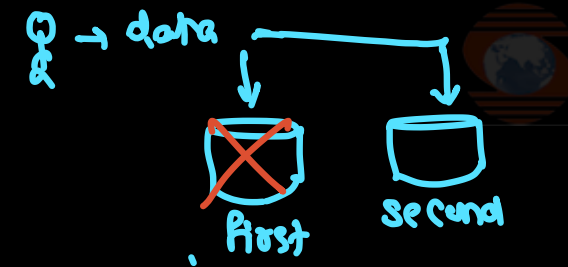
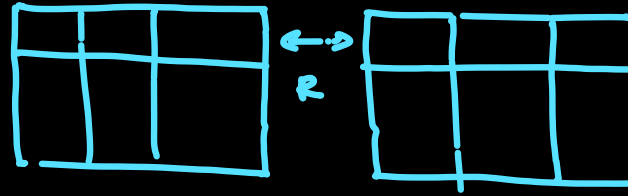
- **Structure:** Data is organized into tables with rows (records) and columns (fields/attributes).
- **Relationships:** Tables are related to each other through keys.
- **Key Concepts:**
 - **Primary Key:** Uniquely identifies each row in a table. (e.g., CustomerID, ProductID)
 - **Foreign Key:** A column in one table that references the primary key of another table. Establishes a link between tables.
 - **Normalization:** The process of organizing data to reduce redundancy and improve data integrity
- **Examples:** MySQL, PostgreSQL, Oracle, Microsoft SQL Server, SQLite

■ Object-Oriented Databases

- Store data as objects, similar to object-oriented programming. (Less common now)

■ In-Memory Databases

- Store data primarily in RAM for very fast access. (e.g., Redis, Memcached) → Cache



Types of Databases

→ loose schema

■ NoSQL Databases (Not Only SQL)

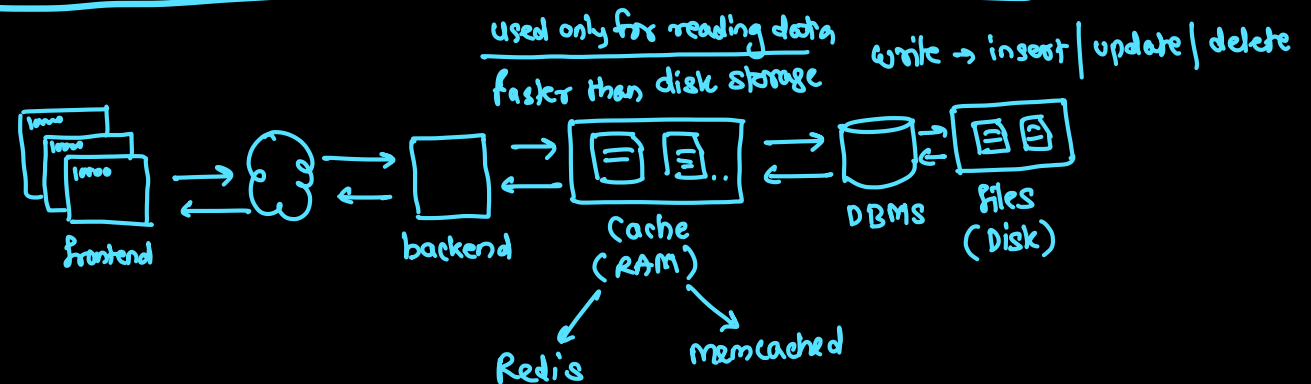
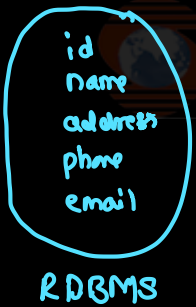
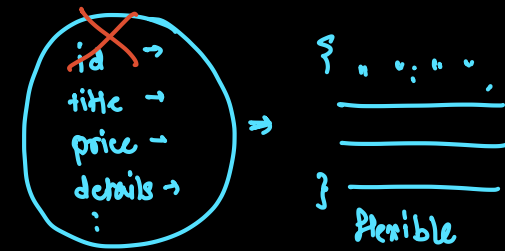
- **Structure:** More flexible than relational databases. Data can be stored in various formats (document, key-value, graph, column family).

- **Scalability:** Designed for handling large volumes of data and high traffic.

↗ Vertical
↘ Horizontal

■ Types (Common):

- **Document Databases:** Store data as JSON-like documents. (e.g., MongoDB) → BSON → binary JSON
- **Key-Value Stores:** Simple storage of key-value pairs. (e.g., Redis)
- **Graph Databases:** Store data as nodes and edges, ideal for representing relationships. (e.g., Neo4j)
- **Column Family Databases:** Store data in columns rather than rows. (e.g., Cassandra)



Database Design → Schema (structure of database)



- Good design is crucial for performance, scalability, and data integrity
- Steps (Relational Databases) *↪ check if flexibility needed*
 - Requirements Gathering: Understand the data to be stored and how it will be used
 - Conceptual Design: Create a high-level model of the data, often using Entity-Relationship Diagrams (ERDs)
 - Entities: Real-world objects or concepts (e.g., Customer, Product, Order) → *tables*
 - Attributes: Characteristics of an entity (e.g., Customer Name, Product Price)
 - Relationships: How entities relate to each other (e.g., a Customer places an Order)
 - Logical Design: Translate the conceptual model into a relational schema (tables, columns, data types)
 - Physical Design: Optimize the database for performance (indexing, storage options)

Customer

- ① name
- ② email
- ③ password
- ④ address : city, zipcode, state..
- ⑤ phone no

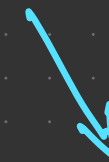
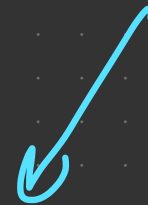
order

- ① customer's information
- ② date
- ③ total amount
- ④ Status
- ⑤ product : title, price, quantities, details...

id	name	email	password	phone	city	zipcode	state
1	cust1	c1@f.com	test	1234	pune	411041	MH
2	cust1	c1@f.com	test	1234	mumbai	400100	MH

customer → bad design as two records are storing same customer information → redundancy

id	name	email	password	phone	city	zipcode	state
1	cust1	cl@f.com	test	1234	pune	411041	MH
2	cust1	cl@f.com	test	1234	Mumbai	400100	MH



relationship

references customers.id

id ^{pk}	name	email	password	phone
1	cust1	cl@f.com	test	1234

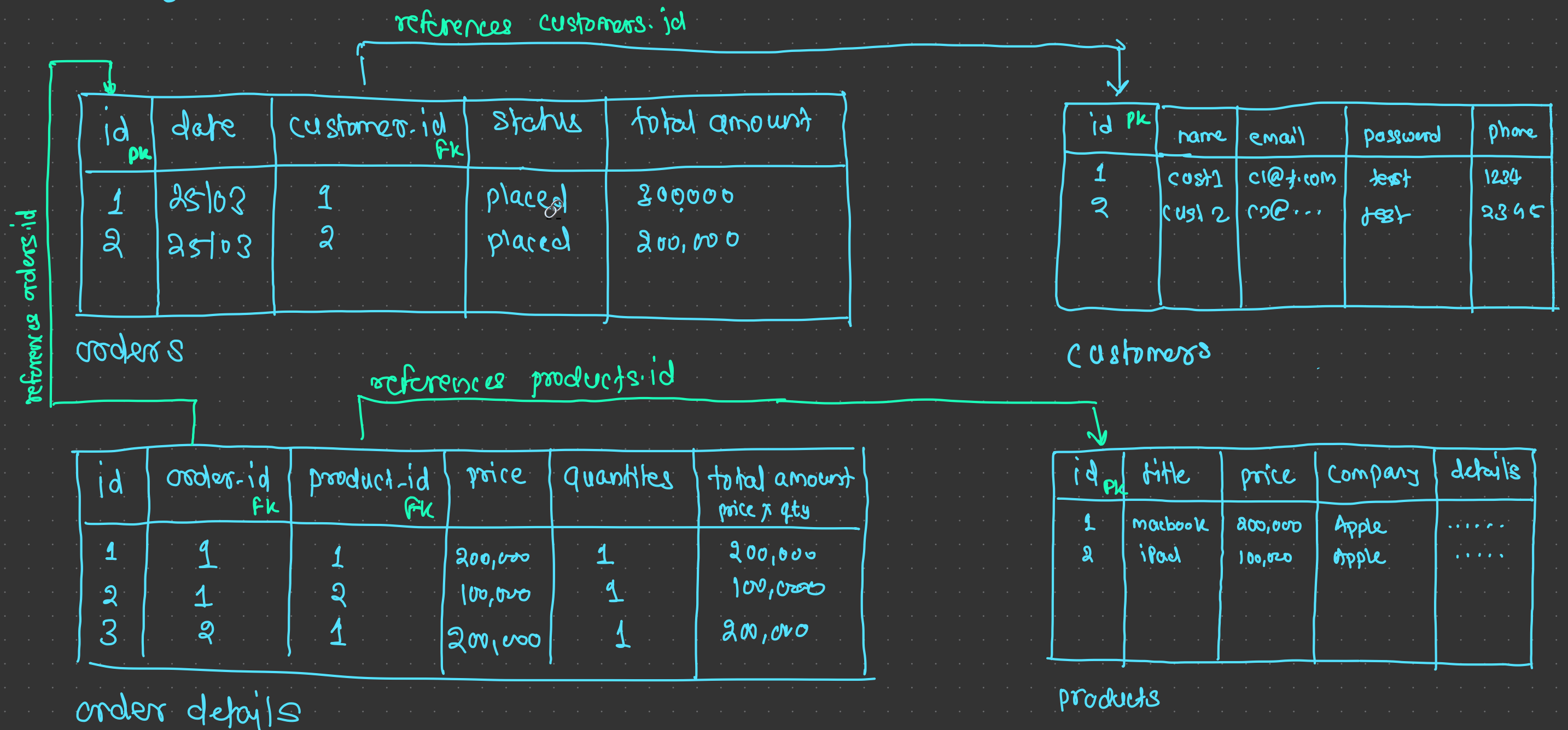
customers

id ^{pk}	city	zipcode	state	customer_id ^{fk}
1	pune	411041	MH	1
2	Mumbai	400100	MH	1

addresses

id	date	total amount	name	email	password	p. title	p. details	p. company	p. price	p. quantities
	25/03	200,000	customer1	cl...	abcd	macbook	macbook details	Apple	200,000	1
	25/03	200,000	customer1	cl...	abcd	ipad	ipad details	Apple	100,000	1
	25/03	200,000	customer2	c2...	xy2	macbook	macbook details	Apple	200,000	1

bad design → redundant data



Normalization



- Reducing data redundancy and improving data integrity
- Common Normal Forms: 1NF, 2NF, 3NF
- 1NF: Eliminate repeating groups of data
- 2NF: Eliminate redundant data that depends on only part of the primary key
- 3NF: Eliminate redundant data that depends on non-key attributes



MySQL



What is MySQL?



- **Relational Database Management System (RDBMS):** MySQL is a popular **open-source** RDBMS. This means it stores data in tables with rows and columns, defining relationships between them
- **Open Source:** It's free to use (with some commercial licensing options for specific features/support).
- **Widely Used:** Powers a significant portion of the web, used by companies like Facebook, Google (for some applications), Wikipedia, and many more
- **Part of Oracle:** MySQL is owned by Oracle Corporation
- **ACID Compliant:** Ensures data integrity through Atomicity, Consistency, Isolation, and Durability

History



- Created by David Axmark and Allan Larsson (Sweden) in 1995
- The initial project was named "Maria" after their daughter. However, due to licensing issues with a similar database system (mSQL), they decided to rename it "MySQL."
- In 1996, MySQL AB released the first version under an open-source license
- From 1997 to 2003, countless applications adopted MySQL, combination of PHP and MySQL became dominant force in web development landscape → LAMP / WAMP / MAMP stack
- In 2003, Sun Microsystems acquired MySQL AB for \$1 billion in cash and stock
- In April 2010, Oracle acquired Sun Microsystems, including MySQL
- Oracle continues to develop and support MySQL, offering both open-source (Community Edition) and commercial editions

Why MySQL is popular?

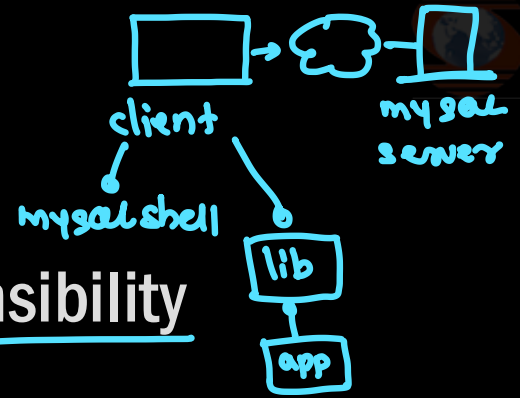


- Fast and high Performance database
- Easy to use, maintain and administer
- Easily available and maintain integrity of database
- Provides scalability, usability and reliability
- Low cost hardware → low infrastructure → machine | internet | resources
- MySQL can read simple and complex queries and write operations
- InnoDB is default and widely used storage engine
- Provides strong indexing support → performance | optimization
- Provides SSL support for secured connections
- Provides powerful data encryption and accuracy
- Provides Cross-platform compatibility

MySQL Architecture

client → mysql

server → mysqld → runs as background process



- It is designed as a series of layers, each responsible for specific tasks
- This layered approach promotes modularity, maintainability, and extensibility

■ Client Layer

- This is the interface through which users and applications interact with MySQL
- It includes:
 - Client Libraries: Libraries like libmysqlclient (C/C++), connectors for PHP, Python, Java, etc. These libraries handle communication with the MySQL server → mysql shell
 - Command-Line Client: The mysql command-line tool for interacting with the server → mysql
 - Graphical User Interfaces (GUIs): Tools like MySQL Workbench, phpMyAdmin, DBeaver

■ Connection Layer

- Manages client connections to the server
- Handles authentication (verifying user credentials) and authorization (determining what a user is allowed to do)

MySQL Architecture



■ SQL Layer

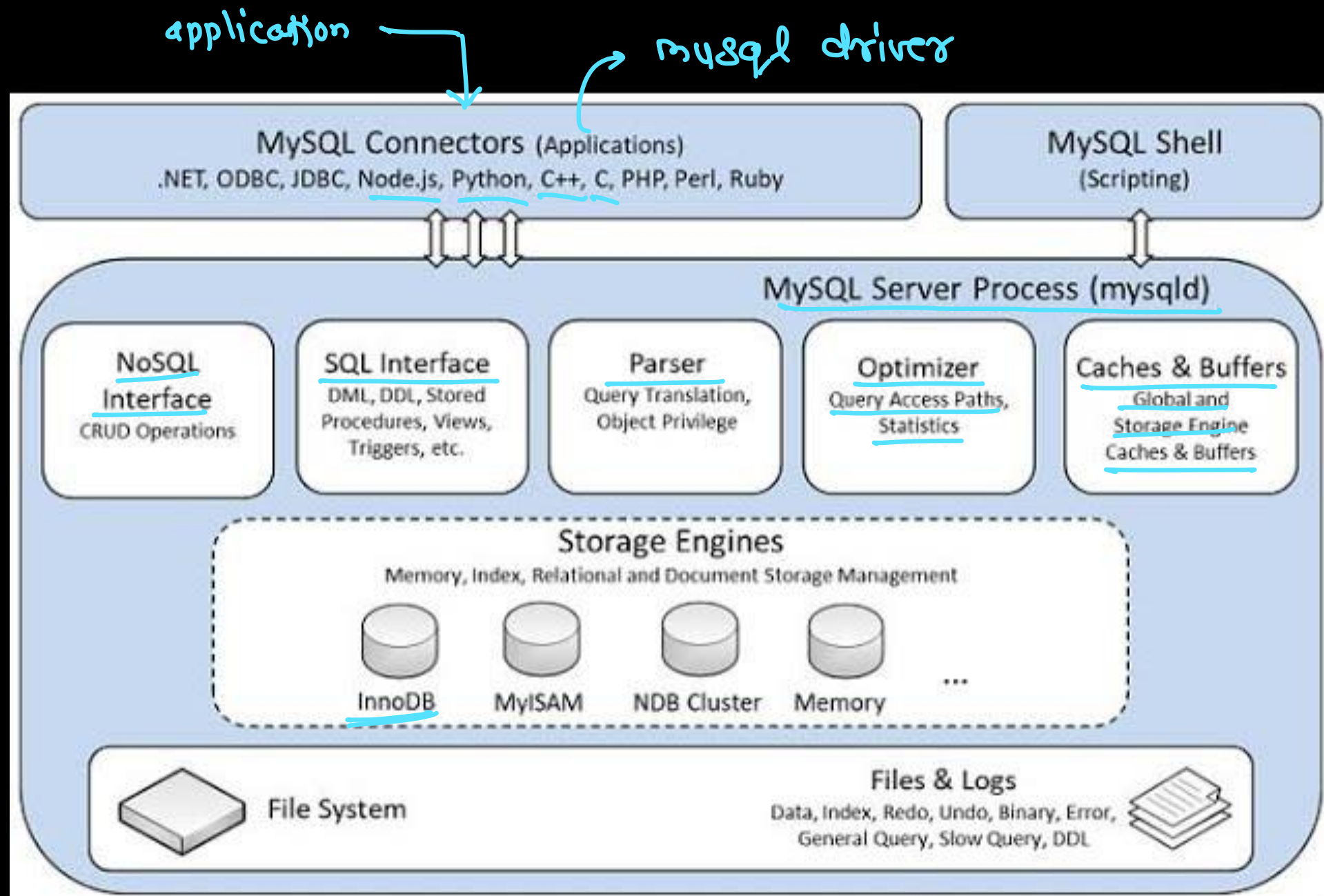
- The heart of MySQL. This layer parses SQL statements, validates them against the database schema (table definitions, data types), and optimizes query execution plans

■ Storage Engine Layer

- This is where the actual data storage and retrieval happens
- It's responsible for interacting with the underlying file system to read and write data
- This layer is pluggable, meaning you can use different storage engines

■ File System Layer

- The lowest layer, directly interacting with the operating system's file system to store and retrieve data files



SQL Layer



■ Query Optimizer

↪ Query plan → plan of executing query

- A critical part of the SQL layer which analyzes SQL queries and determines the most efficient way to execute them, considering indexes, table statistics, and join algorithms

■ SQL Parser

- Takes the SQL query string from the client, analyzes its syntax, and converts it into an internal representation that can be understood by other components

■ Transaction Manager

- Ensures ACID properties (Atomicity, Consistency, Isolation, Durability) by managing transactions. It coordinates read and write operations to maintain data integrity.

■ Lock Manager

- Controls access to database resources (tables, rows) to prevent conflicts and ensure data consistency during concurrent operations.

■ Buffer Pool

- A cache that stores frequently accessed data pages (blocks of data) in memory. This significantly improves performance by reducing the need to read from disk

Pluggable Architecture



- One of MySQL's key strengths is its pluggable architecture, particularly the ability to swap out storage engines
- This allows you to choose the best engine for your specific needs
- Engines supported
 - ✓ InnoDB → default
 - ✓ Memory
 - ✓ MyISAM
 - ✓ Federated
 - ✓ ndbinfo
 - ✓ Blackhole
 - ✓ CSV
 - ✓ Archive
 - ✓ ndbCluster

Data Files & Directory Structure

mysql-home



- MySQL stores data in a directory structure, typically under /var/lib/mysql (on Linux systems)
- The key files include:
- .frm Files: Contain table definitions (schema information)
- .ibd Files (InnoDB): Contain data and indexes for InnoDB tables
- .myi Files (MyISAM): Contain index information for MyISAM tables
- .myd Files (MyISAM): Contain data for MyISAM tables
- mysql.log files: Binary log files used for replication and point-in-time recovery



SQL



What is SQL?



- SQL is a standard programming language designed for managing and manipulating data held in relational database management systems (RDBMS)
- It's not a programming language like Python or Java; it's a query language
- SQL operates on data organized in tables with rows (records) and columns (fields)
- The relationships between these tables are defined, allowing for efficient data storage and retrieval
↓
Schema
- SQL allows you to:
 - Query Data (SELECT): Retrieve specific data from one or more tables.
 - Insert Data (INSERT): Add new records to a table.
 - Update Data (UPDATE): Modify existing data in a table.
 - Delete Data (DELETE): Remove records from a table.
 - Define Data (CREATE, ALTER, DROP): Create, modify, and delete database objects like tables, indexes, views, etc.
 - Control Access (GRANT, REVOKE): Manage user permissions and security within the database.

History



- **1970s - 1980s: Origins at IBM:** SQL (originally SEQUEL – Structured English Query Language) was developed by Donald D. Chamberlin and Raymond F. Boyce at IBM in 1974. It was designed to simplify data access for System R, an experimental relational database system.
- **ANSI/ISO Standardization (1980s):** The ANSI (American National Standards Institute) and ISO (International Organization for Standardization) began standardizing SQL, leading to the first official standard in 1986 (SQL-86).
- **Evolution of Standards:** Subsequent standards like SQL-92, SQL-99, SQL:2003, and SQL:2011 have introduced new features and refinements.
- **MySQL's Emergence:** MySQL was created in 1995. It quickly gained popularity as a free, open-source alternative to commercial database systems.

Advantages of Using SQL



- **Data Integrity**: Relational databases enforce data integrity through constraints, relationships, and transactions.
- **Data Consistency**: Transactions ensure that data modifications are either fully completed or rolled back, maintaining consistency.
- **Flexibility**: Allows for complex queries to retrieve and manipulate data in various ways.
- **Scalability**: MySQL can handle large volumes of data and high traffic loads
- **Security**: MySQL provides robust security features to protect data from unauthorized access.
- **Portability**: SQL is a standard, so your skills and queries can be adapted to other RDBMS systems.
- **Open Source (MySQL)**: Cost-effective and allows for community support and customization.

SQL Command Types



- **Data Definition Language (DDL)**: Defines the database **schema**
 - **CREATE**: Creates a new entity like table, database, function, stored procedure
 - **ALTER**: Modifies an existing entity's structure
 - **DROP**: Deletes an existing entity
 - **TRUNCATE**: Removes all rows from a table
- **Data Manipulation Language (DML)**: Manipulates the data within tables
 - **INSERT**: Adds new rows to a table
 - **UPDATE**: Modifies existing data in a table
 - **DELETE**: Removes rows from a table
- **Data Query Language (DQL)**: Selects the data from the tables
 - **SELECT**: Retrieves data from a table or multiple tables.

SQL Command Types



- **Data Control Language (DCL):** Controls access to data and database objects
 - GRANT: Assigns privileges to users or roles
 - REVOKE: Removes privileges from users or roles
- **Transaction Control Language (TCL):** Manages transactions to ensure data consistency.
 - START TRANSACTION: Begins a transaction
 - COMMIT: Saves changes made during the transaction
 - ROLLBACK: Discards changes made during the transaction
 - SAVEPOINT: Creates a save point within current transaction



Database

Database

table | view | trigger . .

- At its core, a database is an organized collection of data
- A database is a structured way to store information electronically. This information can be anything – customer details, product inventory, website content, etc.
- In MySQL, a "database" is a logical grouping of tables and related objects
- **Directory Structure:** On the file system, each MySQL database is typically represented by a directory
- **Data Files:** Within that directory, you'll find several files:
 - .frm files: These store the table
 - .ibd files: These store the actual data for your tables
 - Other files: files related to indexes, logs, and other database-specific information
- **Metadata:** MySQL maintains a system catalog (metadata) that stores information about the database itself

Creating Database



- **Basic Creation**

```
CREATE DATABASE database_name;
```

- database_name: The name you want to give your database

- **Creation with IF NOT EXISTS**

```
CREATE DATABASE IF NOT EXISTS database_name;
```



Deleting Database

- **Basic Deletion**

```
DROP DATABASE database_name;
```

- **Deletion with IF EXISTS**

```
DROP DATABASE IF EXISTS database_name;
```

Using Database



```
USE database_name;
```

- This tells MySQL that you want to work with this database for subsequent queries
- All table creation, data manipulation (SELECT, INSERT, UPDATE, DELETE), etc., will now operate on the specified database.



Table

What is a table ?



- At its core, a table in MySQL (and relational databases generally) is the primary way you organize and store data
- A table represents a specific entity or category of information
- For example:
 - A "customers" table would store data about your customers (name, address, email, etc.)
 - A "products" table would store data about the products you sell (name, price, description, etc.)
 - An "orders" table would store data about customer orders (order date, items ordered, etc.)
- The key is that a table has a defined structure:
 - **Columns (Fields):**
 - These define the attributes or characteristics of the data you're storing. Each column has a name and a specific data type (e.g., integer, text, date, decimal).
 - **Rows (Records):**
 - Each row represents a single instance or record of the entity. For example, in a "customers" table, each row would represent one customer.

Data Types Supported - Numeric



■ Integers

- TINYINT: Very small integer (signed) - Range: -128 to 127
- SMALLINT: Small integer (signed) - Range: -32,768 to 32,767
- MEDIUMINT: Medium integer (signed) - Range: -8,388,608 to 8,388,607
- INT or INTEGER: Standard integer (signed) - Range: -2,147,483,648 to 2,147,483,647
- BIGINT: Large integer (signed) - Range: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
- UNSIGNED TINYINT, UNSIGNED SMALLINT, UNSIGNED MEDIUMINT, UNSIGNED INT, UNSIGNED BIGINT: Unsigned versions of the above (only positive values)

■ Floating-Point Numbers

- FLOAT: Single-precision floating-point number. (Typically 4 bytes)
- DOUBLE or DOUBLE PRECISION: Double-precision floating-point number. (Typically 8 bytes)
- DECIMAL(M, D) or NUMERIC(M, D): Fixed-point number. M is the total number of digits (precision), and D is the number of digits after the decimal point (scale). This is ideal for financial calculations where exact precision is required.

Data Types Supported



■ String/Text Data Types

- CHAR(length): Fixed-length string. Pads with spaces if the string is shorter than length.
- ✓ VARCHAR(length): Variable-length string. Stores only the characters actually used, up to length.
- TEXT: Long text string. Can store up to 65,535 characters.
- MEDIUMTEXT: Longer text string (up to 16,777,215 characters).
- LONGTEXT: Very long text string (up to 4,294,967,295 characters).
- ENUM('value1', 'value2', ...): A string that can only be one of the specified values.
- SET('value1', 'value2', ...): A string that can be a combination of the specified values.

■ Spatial Data Types (MySQL 5.7 and later)

- GEOMETRY: Generic spatial data type.
- POINT: Represents a single point in space.
- LINESTRING: Represents a line segment.
- POLYGON: Represents a polygon.
- GEOMETRYCOLLECTION: A collection of geometry objects.

Data Types Supported



■ Date & Time Data Types

- DATE: Stores a date (YYYY-MM-DD)
- TIME: Stores a time (HH:MI:SS)
- DATETIME: Stores both date and time (YYYY-MM-DD HH:MI:SS)
- TIMESTAMP: Stores a timestamp, automatically updated when the row is modified. Range: 1970-01-01 00:00:01 UTC to 2038-01-19 03:14:07 UTC (for older MySQL versions). Newer versions have extended the range
- YEAR: Stores a year (YYYY or YYYY-YYYY)

■ JSON Data Type (MySQL 5.7 and later)

- JSON: Stores data in JSON format. Allows for efficient querying and manipulation of JSON documents.



DDL

SQL Commands



■ Basic Table Creation

```
CREATE TABLE my_database.customers (  
    customer_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(255) NOT NULL,  
    last_name VARCHAR(255),  
    email VARCHAR(255) UNIQUE,  
    registration_date DATE  
);
```

■ Creating a Table with CHECK Constraint

```
CREATE TABLE my_database.products (  
    product_id INT PRIMARY KEY AUTO_INCREMENT,  
    price DECIMAL(10, 2) CHECK (price > 0) --  
);
```

■ Creating a Table with a Foreign Key

```
CREATE TABLE my_database.orders (  
    order_id INT PRIMARY KEY AUTO_INCREMENT,  
    customer_id INT, -- Foreign key referencing customers table  
    order_date DATE,  
    total_amount DECIMAL(10, 2),  
    FOREIGN KEY (customer_id) REFERENCES my_database.customers(customer_id)  
);
```

SQL Commands



- **Showing Table Structure**

```
DESCRIBE my_database.customers;
```

- **Showing Create Table Statement**

```
SHOW CREATE TABLE my_database.customers;
```

- **Listing Tables in a Database**

```
SHOW TABLES;
```

- **Listing Tables with a Specific Pattern**

```
SHOW TABLES LIKE 'cust%';
```

- **Renaming a Table**

```
RENAME TABLE my_database.old_table TO my_database.new_table;
```

SQL Commands



- Dropping a Table: (This permanently deletes the table and all its data)

```
DROP TABLE my_database.customers;
```

- Truncating a Table: (Removes all rows but keeps the table structure. Faster than DELETE without a WHERE clause, but cannot be rolled back.)

```
TRUNCATE TABLE my_database.orders;
```

SQL Commands



- **Adding a New Column**

```
ALTER TABLE my_database.customers ADD COLUMN phone_number VARCHAR(20);
```

- **Modifying a Column's Data Type**

```
ALTER TABLE my_database.customers MODIFY COLUMN email VARCHAR(300);
```

- **Adding a Constraint (e.g., NOT NULL)**

```
ALTER TABLE my_database.customers ADD CONSTRAINT chk_last_name CHECK (last_name IS NOT NULL);
```

- **Adding a Foreign Key Constraint: (If not defined during table creation)**

```
ALTER TABLE my_database.orders ADD CONSTRAINT fk_customer_id FOREIGN KEY (customer_id) REFERENCES  
my_database.customers(customer_id);
```

- **Renaming a Column**

```
ALTER TABLE my_database.customers RENAME COLUMN first_name TO given_name;
```

- **Dropping a Column**

```
ALTER TABLE my_database.customers DROP COLUMN phone_number;
```



DML

insert | update | delete

Insert Data



- Inserting single records

```
INSERT INTO customers (customer_id, first_name, last_name, email)  
VALUES (1, 'John', 'Doe', 'john.doe@example.com');
```

- Inserting multiple records (bulk insert)

```
INSERT INTO customers (customer_id, first_name, last_name, email)  
VALUES  
(2, 'Jane', 'Smith', 'jane.smith@example.com'),  
(3, 'Alice', 'Johnson', 'alice.johnson@example.com');
```

- Using INSERT IGNORE

```
INSERT IGNORE INTO customers (customer_id, first_name, last_name, email)  
VALUES (1, 'John', 'Doe', 'john.doe@example.com'); -- This will be ignored if  
customer_id 1 already exists.
```

- Using INSERT ON DUPLICATE KEY UPDATE

```
INSERT INTO customers (customer_id, first_name, last_name, email)  
VALUES (1, 'John', 'Doe', 'john.doe@example.com')  
ON DUPLICATE KEY UPDATE first_name = VALUES(first_name), last_name =  
VALUES(last_name);
```

Update Data



- Basic update query

where clause → filtering records

```
UPDATE customers SET email = 'new_email@example.com' WHERE customer_id = 1;
```

- Update multiple fields

```
UPDATE customers SET first_name = 'John', last_name = 'Doe' WHERE customer_id = 2;
```

- Increment a field

```
UPDATE customers SET customer_id = customer_id + 1;
```

- Update existing value

```
UPDATE customers SET email = LOWER(email);
```

Delete data



- Delete a specific record

```
DELETE FROM customers WHERE customer_id = 1;
```

- Delete all records

```
DELETE FROM customers;
```

- Truncate table

```
TRUNCATE TABLE my_database.orders;
```



DQL



Basic Retrieval



■ Select all columns from a table

```
SELECT * FROM customers;
```

projection → list of columns → mandatory
selection → filtering records (rows)

select <projection> from <table name> where <selection>
↓
* = all columns

■ Select specific columns

```
SELECT first_name, last_name FROM customers;
```

■ Alias a column

```
SELECT first_name AS given_name, last_name AS family_name FROM customers;
```

Filtering & Sorting



■ Filtering with WHERE

```
SELECT * FROM customers WHERE customer_id = 1;  
SELECT * FROM products WHERE price > 100;
```

condition

■ Using AND and OR

```
SELECT * FROM customers WHERE city = 'New York' AND state = 'NY';  
SELECT * FROM orders WHERE order_date > '2023-01-01' OR total_amount > 500;
```

■ Using LIKE for pattern matching

any character

```
SELECT * FROM customers WHERE last_name LIKE 'S%'; -- Last name starting with "S"  
SELECT * FROM products WHERE description LIKE '%keyword%'; -- Products containing the keyword
```

■ Using IN to check against multiple values

```
SELECT * FROM customers WHERE city IN ('New York', 'Los Angeles', 'Chicago');
```

■ Using BETWEEN to check within a range

```
SELECT * FROM products WHERE price BETWEEN 50 AND 100;
```

■ Sorting with ORDER BY

```
SELECT * FROM customers ORDER BY last_name ASC; -- Ascending order by last name  
SELECT * FROM products ORDER BY price DESC, product_name ASC; -- Descending order
```

Aggregate Functions



- **COUNT()**: Counts rows or values

```
SELECT COUNT(*) FROM customers; -- Total number of customers  
SELECT COUNT(DISTINCT city) FROM customers; -- Number of unique cities
```

- **SUM()**: Calculates the sum of values

```
SELECT SUM(total_amount) FROM orders; -- Total revenue from all orders
```

- **AVG()**: Calculates the average of values

```
SELECT AVG(price) FROM products; -- Average product price
```

- **MIN()**: Finds the minimum value

```
SELECT MIN(price) FROM products; -- Lowest product price
```

- **MAX()**: Finds the maximum value

```
SELECT MAX(price) FROM products; -- Highest product price
```

- **GROUP BY**: Groups rows based on one or more columns

```
SELECT city, COUNT(*) FROM customers GROUP BY city; -- Count customers per city
```

- **HAVING**: Filters grouped data (used with GROUP BY)

```
SELECT city, COUNT(*) FROM customers GROUP BY city HAVING COUNT(*) > 10;
```

Subqueries (Nested Queries)



- **Subquery in WHERE clause**

```
SELECT * FROM orders WHERE customer_id IN (SELECT customer_id FROM customers WHERE city = 'New York');
```

- **Subquery in SELECT clause: (Less common, but possible)**

```
SELECT order_id, total_amount, (SELECT customer_name FROM customers WHERE customer_id = orders.customer_id)  
AS customer_name FROM orders;
```

- **EXISTS and NOT EXISTS: Checks for the existence of rows**

```
SELECT * FROM customers WHERE EXISTS (SELECT 1 FROM orders WHERE orders.customer_id =  
customers.customer_id); -- Customers who have placed at least one order
```

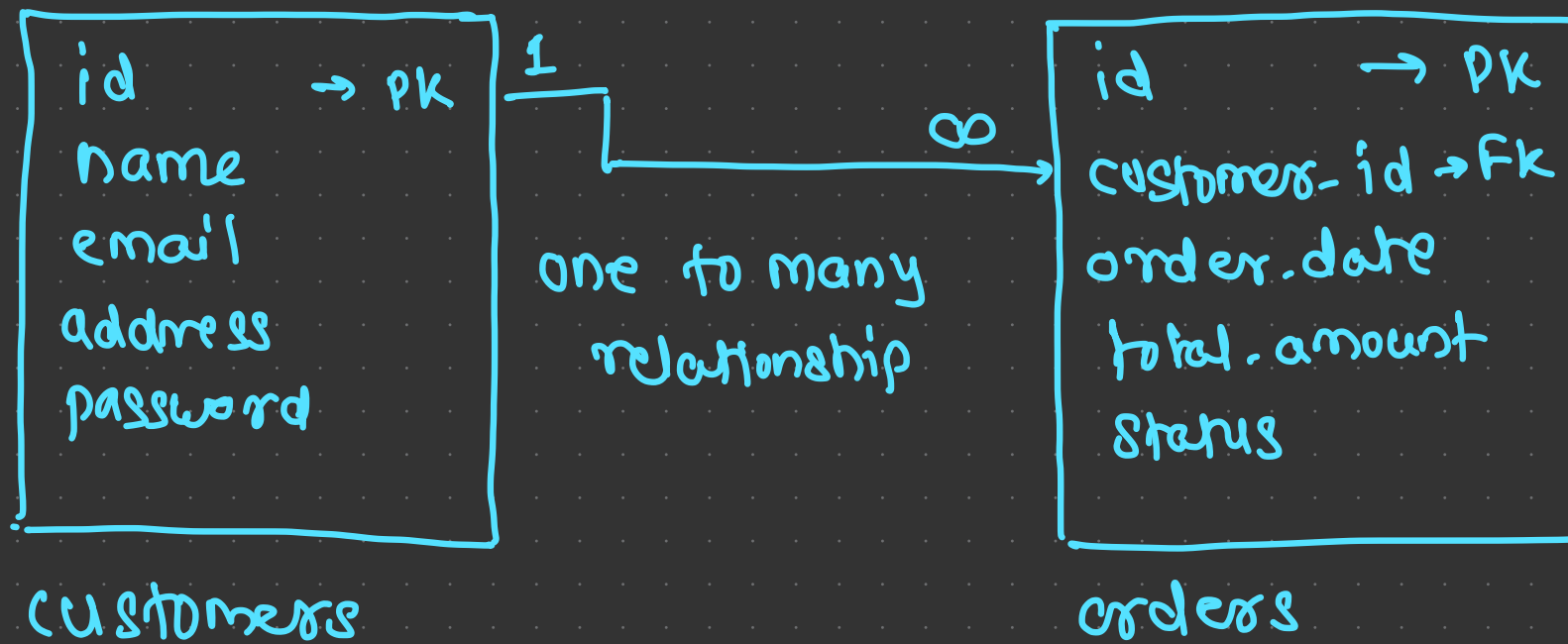
Joins (Combining Tables)

→ RDBMS



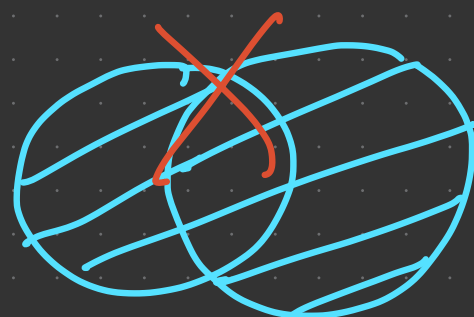
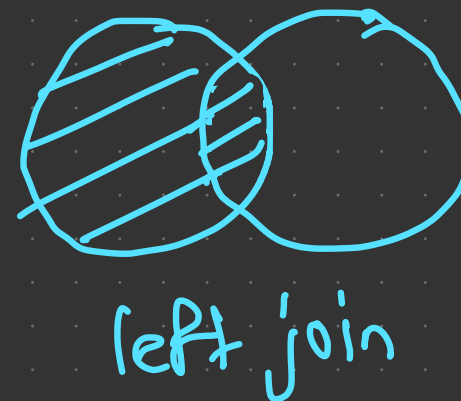
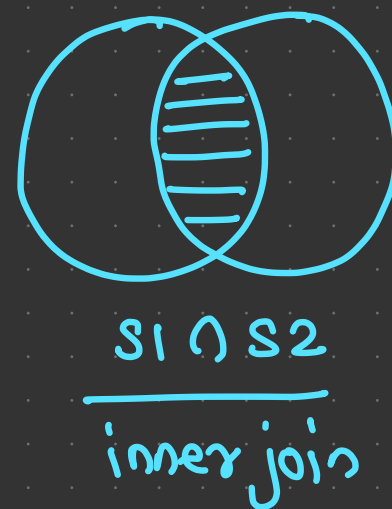
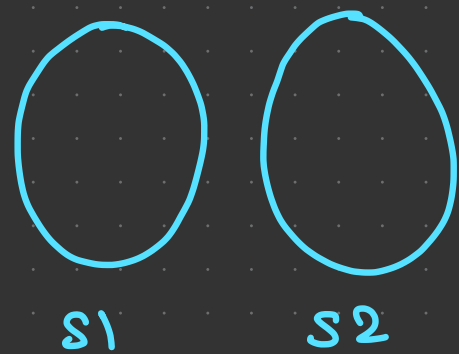
- Joins are a fundamental aspect of SQL, allowing you to combine rows from two or more tables based on a related column between them
- The most common types of joins are:
 - **INNER JOIN**: Returns only the rows that have matching values in both tables
 - **LEFT OUTER JOIN (or simply LEFT JOIN)**: Returns all rows from the left table, and the matched rows from the right table. If there is no match, the result is NULL on the side of the right table
 - **RIGHT OUTER JOIN (or simply RIGHT JOIN)**: Returns all rows from the right table, and the matched rows from the left table. If there is no match, the result is NULL on the side of the left table
 - **FULL OUTER JOIN**: Returns all rows when there is a match in either the left or right table. If there is no match, the result is NULL on one side (not supported in MySQL)

→ common rows



Relationship types

- one to one
- one to many
- many to one
- many to many



Full outer join

Joins (Combining Tables)



Common Rows

- **INNER JOIN:** Returns rows only when there's a match in both tables

```
SELECT orders.order_id, customers.first_name, customers.last_name FROM orders INNER JOIN customers ON  
orders.customer_id = customers.customer_id;
```

- **LEFT JOIN:** Returns all rows from the left table and matching rows from the right table. If there's no match, it returns NULL for the right table columns

```
SELECT customers.*, orders.* FROM customers LEFT JOIN orders ON customers.customer_id = orders.customer_id;
```

- **RIGHT JOIN:** Returns all rows from the right table and matching rows from the left table

```
SELECT customers.*, orders.* FROM customers RIGHT JOIN orders ON  
customers.customer_id = orders.customer_id;
```



DCL

Data Control Language



■ Creating a User

```
CREATE USER 'cust_user'@'localhost' IDENTIFIED BY 'password123';
```

■ Granting Permissions

```
GRANT SELECT, INSERT ON customers TO 'cust_user'@'localhost';
```

■ Common Privileges

- **ALL PRIVILEGES**: Grants all possible privileges to the user
- **SELECT**: Allows the user to read data from tables
- **INSERT**: Allows the user to insert new rows into tables
- **UPDATE**: Allows the user to update existing rows in tables
- **DELETE**: Allows the user to delete rows from tables

■ Revoke Privileges

```
REVOKE SELECT ON customers FROM 'cust_user'@'localhost';
```

■ Apply Privileges

```
FLUSH PRIVILEGES;
```

■ Checking Permissions

```
SHOW GRANTS FOR 'cust_user'@'localhost';
```



Functions

MySQL Functions



- MySQL provides a variety of **built-in functions** that can be used within SQL queries to perform specific tasks, such as data manipulation, date and time calculations, string operations, and more
- These functions help in enhancing the functionality of queries by allowing for complex data processing directly within the **database server**



Benefits of function

- Code Reusability
- Enhanced Readability
- Performance Optimization
- Encapsulation of Logic
- Data Abstraction
- Security and Access Control
- Complex Calculations
- Dynamic Data Handling
- ~~Dynamic Data Handling~~
- Integration with Applications

MySQL Built-in Functions



- **String Functions**: Used to manipulate strings
 - Examples: CONCAT(), SUBSTRING(), LOWER(), UPPER()
- **Numeric Functions**: Used to perform arithmetic operations or return values based on numeric data.
 - Examples: SUM(), AVG(), ROUND(), ABS()
- **Date and Time Functions**: Used to manipulate date and time values.
 - Examples: NOW(), DATE_ADD(), DATEDIFF()
- **Aggregation Functions**: Used to perform calculations on a set of values and return a single value.
 - Examples: COUNT(), SUM(), AVG(), MAX(), MIN()
- **Control Flow Functions**: Used to perform conditional operations.
 - Examples: IF(), CASE
- **Information Functions**: Used to retrieve information about the database or table.
 - Examples: DATABASE(), VERSION()
- **System Functions**: Used to interact with the MySQL server or environment.
 - Examples: SLEEP(), USER()

User Defined functions

- MySQL also allows the creation of user-defined functions (UDFs), which are stored procedures that can be called like regular SQL functions
- UDFs can enhance the reusability of code and provide more complex logic

DELIMITER // → used to End function definition

CREATE FUNCTION calculate_discount(price DECIMAL(10, 2), discount_rate DECIMAL(5, 2))

RETURNS DECIMAL(10, 2)

DETERMINISTIC → always return same result for same input

BEGIN

DECLARE final_price DECIMAL(10, 2);

SET final_price = price - (price * discount_rate / 100);

RETURN final_price;

END //

DELIMITER ;

SELECT calculate_discount(100.00, 10) AS discounted_price;

Square(2) = 4

RAND()



Stored Procedure

What is Stored Procedure



- Stored procedures are pre-compiled collections of SQL statements stored in the database that can be executed as a single unit
- They offer several advantages, including code reusability, improved performance through reduced network traffic, and enhanced security control
- Unlike functions, stored procedures can execute multiple SQL statements and perform complex operations
- They are compiled once and stored in the database, making them reusable and efficient

Advantages of Stored Procedures



- **Reusability**: Reduces redundancy by allowing developers to reuse the same set of SQL statements multiple times.
- **Performance**: Enhances performance by reducing network traffic between the application and the database server. The procedure is executed on the server side.
- **Security**: Provides an additional layer of security by controlling access to underlying data and operations.
- **Encapsulation**: Abstracts complex business logic from the application, promoting modular design.
- **Consistency**: Ensures that operations are performed consistently across different applications and environments.



Limitations of Stored Procedures

- **Portability**

- Not all SQL databases support stored procedures in the same way, which can affect portability

- **Complexity**

- Overly complex stored procedures can be difficult to debug and maintain

- **Overuse**

- Relying heavily on stored procedures can lead to bloated database logic, potentially separating business rules from the application layer

Create and use Stored Procedure



```
DELIMITER //
```

```
CREATE PROCEDURE GetCustomerDetails(IN customerID INT)
```

```
BEGIN
```

```
    SELECT * FROM customers WHERE customer_id = customerID;
```

```
END //
```

```
DELIMITER ;
```

```
CALL GetCustomerDetails(1);
```



Parameters in Stored Procedures

- Stored procedures can have three types of parameters:
 - IN: Passed from the caller to the procedure
 - OUT: Passed from the procedure to the caller
 - INOUT: Can be passed both ways

Conditional Statements in Stored Procedures



```
DELIMITER //
```

```
CREATE PROCEDURE CheckCustomerBalance(IN customerID INT)
BEGIN
    DECLARE balance DECIMAL(10, 2);

    SELECT account_balance INTO balance FROM accounts WHERE customer_id = customerID;

    IF balance < 0 THEN
        SELECT 'Balance is negative!' AS message;
    ELSE
        SELECT 'Balance is positive.' AS message;
    END IF;
END //
```

```
DELIMITER ;
```



SQL Injection

SQL Injection



- SQL Injection is a security vulnerability
- Occurs when user input is directly used in SQL queries
- Allows attackers to manipulate database queries
- One of the most common web application attacks
- How does it work
 - Application takes user input
 - Input is inserted into SQL query
 - No validation / sanitization
 - Attacker injects malicious SQL



Types of SQL Injection

■ Authentication Bypass

- Use: ' OR 1=1 -- *or { ' OR '1'='1 }*
- Bypasses login system
- Returns all records

■ UNION-Based Injection

- ' UNION SELECT user, password FROM mysql.user --
- Combines query results
- Used to extract sensitive data

■ Blind SQL Injection

- ' AND SLEEP(5) --
- No visible output
- Uses response timing



Impact of SQL Injection

- Unauthorized access
- Data leakage
- Data modification
- Data deletion
- Full database compromise



Prevention Techniques

- Use Parameterized Queries
- Validate input
- Use ORM frameworks
- Apply least privilege → DCL
- Hide error messages

'/.S



Best Practices

- Never trust user input
- Avoid dynamic SQL queries
- Use prepared statements
- Regular security audits → UAPT
- Use WAF (Web Application Firewall)



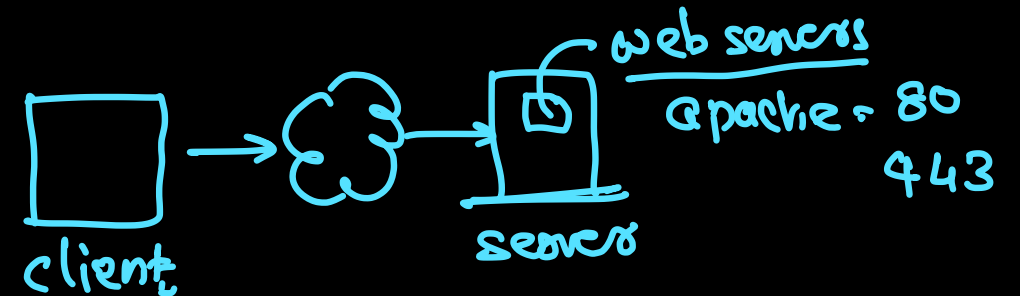
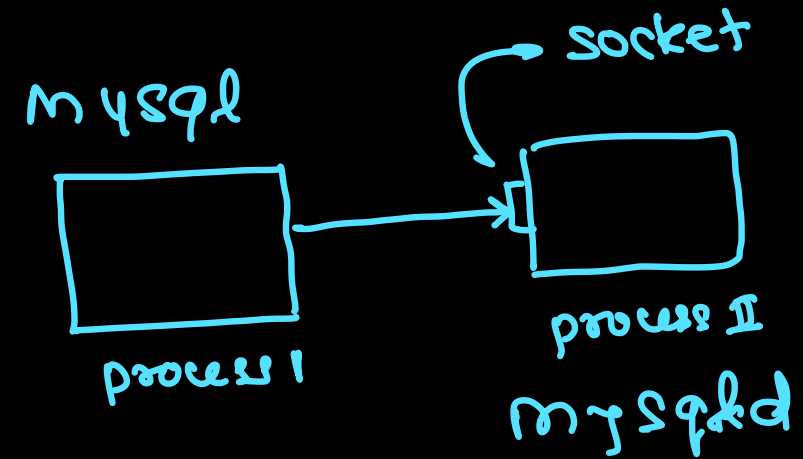
Socket





What is socket

- A socket is an endpoint of communication
- Enables data exchange between
 - Two computers
 - Two processes
 - Works over a network
- Where is it used
 - Chat applications (WhatsApp, Messenger)
 - Online games
 - Web servers
 - Real-time dashboards
 - Stock trading systems





Types of Sockets

■ TCP Socket (SOCK_STREAM)

- Reliable
- Connection-oriented
- Used in web apps

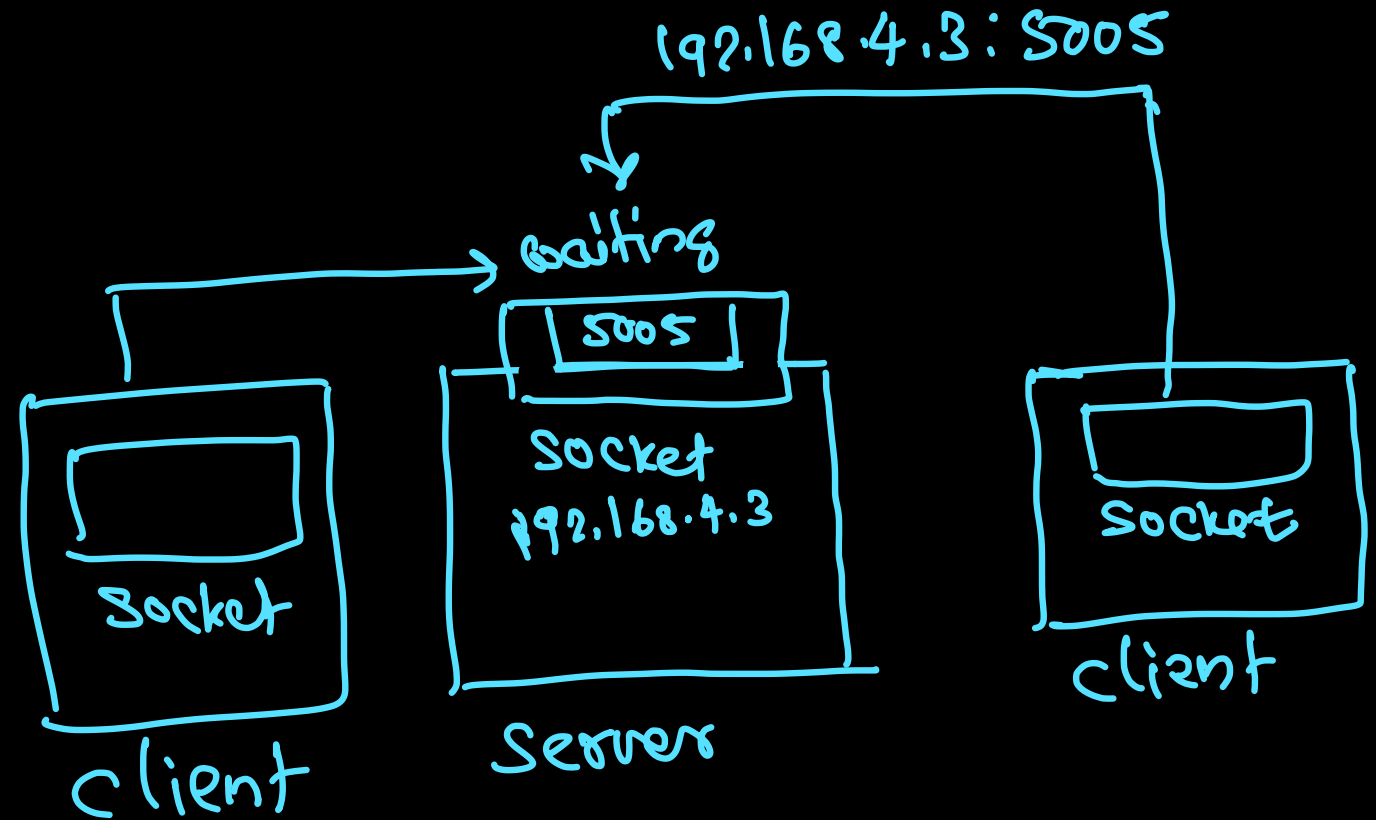
■ UDP Socket (SOCK_DGRAM)

- Faster
- Connectionless
- Used in streaming



Key Socket Functions

- socket(): Create socket
- bind(): Attach IP & port
- listen(): Wait for connections
- accept(): Accept client
- connect(): Client connects
- send(): Send data
- recv(): Receive data



Python Socket Server Code



```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 5000))
server.listen(5) → infinite

print("Server listening...")

client, addr = server.accept() → waits till client connects
print("Connected to:", addr)

msg = client.recv(1024).decode()
print("Client:", msg)

client.send("Hello from server".encode())

client.close()
server.close()
```

Python Socket Client Code



```
import socket  
  
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
client.connect(("127.0.0.1", 5000))  
  
client.send("Hello Server".encode())  
  
msg = client.recv(1024).decode()  
print("Server:", msg)  
  
client.close()
```